

TASK 17 – MONTHLY SALES TREND

Data Analytics Track | Level 1 – Day 17

1. Objective

The objective of this task is to summarize sales data by month and visualize the sales trend using a line chart. This task helps practice date conversion, monthly grouping, aggregation, chronological sorting, and time-series visualization.

2. Tools Used

- Microsoft Excel
- Python
- Pandas
- Matplotlib

3. Dataset

A retail sales dataset was used for this analysis. The dataset contains order-level information such as:

- Order ID
- Order Date
- Region
- Segment
- Product
- Category
- Quantity
- Discount
- Sales

The data covers the period from **January 2025 to December 2025**.

4. Methodology

The following steps were performed:

1. Loaded the retail sales dataset.
2. Converted the **Order_Date** column into a proper date format.
3. Created a monthly field from the order date.
4. Grouped the sales records by month.
5. Calculated the total sales for each month.
6. Sorted the months chronologically.

7. Created a monthly sales summary table.
8. Created a line chart to visualize the sales trend.
9. Performed basic observations from the monthly trend.

5. Monthly Sales Summary

Month	Total Sales
Jan-2025	₹534,982.50
Feb-2025	₹368,025.00
Mar-2025	₹572,775.00
Apr-2025	₹480,970.00
May-2025	₹643,130.00
Jun-2025	₹572,110.00
Jul-2025	₹557,905.00
Aug-2025	₹511,705.00
Sep-2025	₹536,160.00
Oct-2025	₹487,095.00
Nov-2025	₹693,340.00
Dec-2025	₹653,555.00

6. Sales Trend Visualization

A **line chart** was used to visualize the monthly sales trend.

The line chart shows how sales change from January 2025 to December 2025. It makes increases, decreases, peaks, and fluctuations over time easier to identify.

Insert the monthly_sales_trend.png file here.

7. Analysis Summary

The total sales for the year were approximately **₹66.12 lakh**.

The highest monthly sales were recorded in **November 2025**, with approximately **₹6.93 lakh** in sales.

The lowest monthly sales were recorded in **February 2025**, with approximately **₹3.68 lakh** in sales.

The monthly sales values fluctuate throughout the year, with relatively higher sales toward the end of the year.

8. Excel Implementation

In Excel, the sales data was organized into a raw-data table. The **Order_Date** column was formatted as a proper date.

A monthly summary was created using the month extracted from the order date. Total sales were calculated for each month.

A **Line Chart** was then created using:

- X-axis: Month
- Y-axis: Total Sales

The Excel workbook contains:

1. **Raw Data**
2. **Monthly Summary**
3. **Dashboard**
4. **Monthly Sales Trend Chart**

9. Python Implementation

Python was used with Pandas for data processing and Matplotlib for visualization.

The main steps were:

- Read the CSV dataset.
- Convert Order_Date to datetime.
- Extract the month.
- Group sales by month.
- Sort the months chronologically.
- Plot the monthly sales using a line chart.

Python Code

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Load the dataset
```

```
df = pd.read_csv("retail_sales_2025.csv", parse_dates=["Order_Date"])
```

```
# Create monthly period
```

```
df["Month"] = df["Order_Date"].dt.to_period("M")
```

```

# Group sales by month
monthly_sales = (
    df.groupby("Month", as_index=False)["Sales"]
        .sum()
        .sort_values("Month")
)

# Create month labels
monthly_sales["Month_Name"] = monthly_sales["Month"].dt.strftime("%b-%Y")

# Display monthly table
print(monthly_sales[["Month_Name", "Sales"]])

# Create line chart
plt.figure(figsize=(10, 5))
plt.plot(
    monthly_sales["Month_Name"],
    monthly_sales["Sales"],
    marker="o"
)

plt.title("Monthly Sales Trend - 2025")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.xticks(rotation=45)
plt.tight_layout()

plt.savefig("monthly_sales_trend.png", dpi=150)
plt.show()

```

10. Interview Questions and Answers

Q1. What is the best chart for showing time trends?

A **line chart** is generally suitable for time trends because it clearly shows how a value changes over a continuous period.

Q2. Why can date formatting cause errors?

If dates are stored as **text instead of actual date values**, Excel or Python may sort them incorrectly or fail to group them properly. Converting the values into proper date/datetime format prevents these problems.

Q3. Why should months be sorted chronologically?

Months should be sorted chronologically so that the chart represents the actual sequence of time. Otherwise, months may appear in alphabetical order instead of January, February, March, and so on.

Q4. Why do we group sales by month?

Grouping sales by month converts detailed transaction-level data into a summarized view. This makes it easier to identify trends, fluctuations, and high- or low-sales periods.

Q5. Why is a line chart better than a pie chart for this task?

A line chart is designed to show changes over time. A pie chart is mainly used to show proportions of a whole and is not suitable for displaying a time-based trend.

11. Key Learning Outcomes

Through this task, I learned:

- How to handle date columns correctly.
- How to group data by month.
- How to calculate monthly sales totals.
- How to sort dates chronologically.
- How to create line charts in Excel.
- How to perform monthly aggregation using Pandas.
- How to visualize time-based data using Matplotlib.
- How incorrect date formatting can affect analysis.

12. Conclusion

This task successfully demonstrates **monthly sales analysis and time-series visualization** using Excel and Python. The sales data was converted into a monthly summary, sorted chronologically, and represented using a line chart. The analysis makes it easier to understand sales fluctuations throughout the year and demonstrates important data analytics concepts such as date handling, aggregation, and visualization.